Semtracar Web

Brief técnico para desarrollo asistido por IA: contexto, alcance, tecnologías, backend existente, división del trabajo y reglas de Git.

Campo	Valor
Proyecto	Aplicación web de Semtracar S.A. (casillero internacional EE. UU. → Nicaragua). Asignatura Diseño de Sistemas de Internet, UNI-RUSB, grupo 5T1-SIS-S.
Equipo	Juan Agustín Vega Chávez (coordinador), Alberto José Salgado Castillo, Daviana Paola Lourdes Vela Miranda.
Repositorio	https://github.com/juanvega23/semtracar-web
Diseño (Figma)	https://www.figma.com/design/3PGBuku6jEREw7Zraee5wW/Semtracar
Fuentes	Entregable 1 (requerimientos RF-01…RF-68, RNF-01…RNF-42, modelos UWE), Entregable 2 (arquitectura cliente-servidor + MVC), Entregable 3 (tecnologías y estrategias), código de la app móvil Flutter “Nica Cargo Xpress V2”.
Modo de trabajo	Por turnos secuenciales: Juan → Daviana → Alberto → Juan (cierre). El avance vive en `docs/ESTADO.md` dentro del repo.
Versión	1.1. 30 de octubre de 2026 (modelo de trabajo por relevos).

0. Instrucciones obligatorias para la IA que lea este documento

Este documento es la **fuente de verdad** del proyecto. Si una petición del usuario lo contradice, señala la contradicción y pregunta antes de actuar.

1. **El trabajo es por turnos (relevo).** Un integrante termina su parte, la deja en `main` y el siguiente descarga el repo y continúa (sección 7). Al iniciar, lee en este orden: `AGENTS.md` (raíz del repo), `docs/ESTADO.md` (qué está hecho y qué turno sigue) y este brief. **ESTADO.md manda sobre el avance y las decisiones; este brief manda sobre las reglas.**
2. **Identifica al integrante y su turno.** Confirma quién te está usando (**Juan**, **Alberto** o **Daviana**) y comprueba en ESTADO.md que es su turno y que el turno anterior está **Cerrado**. Si no lo está, detente y avísale. Trabaja solo en las tareas de ese turno (sección 7.2).
3. **Al terminar el turno es obligatorio el protocolo de cierre** (sección 7.4). Sin él, la IA del siguiente integrante no sabrá qué sigue.
4. **Autoría de commits.** Antes del primer commit ejecuta `git config user.name` y `git config user.email` y verifica que correspondan al integrante. Nunca cambies el autor de un commit, nunca hagas commits en nombre de otro integrante, nunca hagas push directo a `main`, nunca uses `--force` en ramas compartidas y nunca reescribas historial ya publicado (sección 8).
5. **No inventes el backend.** No supongas tablas, columnas, funciones ni valores. Consulta `src/types/database.ts` (generado) y `supabase/migrations/`. La sección 4 es un resumen inferido del código móvil; si difiere de los tipos generados, mandan los tipos.
6. **El backend está en producción y lo comparte la app móvil.** No modifiques tablas, políticas RLS ni funciones fuera de una migración revisada por Juan. No borres ni renombres columnas existentes: romperías la app móvil.
7. **Secretos.** En el frontend solo existen `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`. La clave `service_role` nunca se usa en el navegador ni se escribe en el repositorio. `.env.local` no se sube.

8. **Idioma.** Todo texto visible al usuario va en español, con moneda en USD y fechas en `America/Managua`. Nombres de código en inglés. Mensajes de commit en español con prefijo convencional (sección 8.4).
9. **Commits pequeños y frecuentes**, uno por cambio lógico, para que el historial refleje el trabajo real de cada integrante. Evita commits gigantes del tipo "todo el módulo".
10. **Antes de abrir un PR**, ejecuta `npm run lint`, `npm run typecheck`, `npm run test` y `npm run build`, y completa la plantilla de PR. Nunca fusiones el PR del mismo integrante que lo creó.
11. **Ante dudas de negocio** (sección 11, "Pendientes"), pregunta al integrante en lugar de decidir.

1. Contexto del proyecto

Semtracar S.A. asigna a cada cliente un casillero en una bodega de EE. UU., recibe sus compras en línea, las traslada a Nicaragua, gestiona aduana y las entrega en sucursal o a domicilio. Ya existe una **aplicación móvil en Flutter** (en el código se llama `testapp`, marca "Nica Cargo Xpress") con backend en **Supabase** (PostgreSQL, Auth, Realtime, Edge Functions).

Objetivo: construir una **aplicación web en React + TypeScript** que use el **mismo proyecto Supabase** (mismo esquema, mismas políticas RLS, mismos RPC) que la app móvil, sin duplicar reglas de negocio (RNF-21). La web es para **clientes, administradores y encargados de almacén**. El repartidor sigue usando únicamente la app móvil.

Arquitectura documentada: cliente-servidor (navegador React ↔ Supabase), organizada internamente con un **MVC adaptado a React** y módulos por dominio (Entregables 2 y 3). La seguridad es de doble capa: la interfaz oculta o bloquea, pero la protección real está en las políticas RLS de PostgreSQL (RNF-01).

2. Alcance de la web

Rol (users.role)	¿En la web?	Qué hace en la web
cliente	Sí	Registro, cotizador, seguimiento de paquetes, recibos y pagos (QR y PDF), solicitud de delivery, avisos y perfil.
admin	Sí	Paquetes y sus 4 estados, facturación y pagos, validación de retiro en sucursal, pedidos de delivery, dashboard y reportes, usuarios y roles, exportación CSV.
almacen	Sí (panel admin restringido)	Asignar paquetes nuevos y estados "Recibido"/"En tránsito". Pedidos y Facturación visibles con candado. Sin estado de pago.
repartidor	**NO**	Solo app móvil. Si inicia sesión en la web se muestra la pantalla **"Tu rol usa la aplicación móvil"** con una breve explicación y un botón para salir. Se cierra la sesión (`supabase.auth.signOut()`). No ve ningún panel ni dato.

2.1 Fuera de alcance (no implementar en la web)

- **Mapa de ruta y ubicación del repartidor** (`DeliveryRouteMap`, columnas `deliverer_lat/lng`). Decisión del equipo: no se lleva a web.
- **Funciones operativas del repartidor:** aceptar o rechazar pedidos, GPS, foto de prueba y completar entrega. La web **no debe llamar** a `accept_delivery`, `complete_delivery`, `update_deliverer_location` ni `delivery_balance`.
- **Recepción de notificaciones push (FCM)** y tabla `user_push_tokens`. En la web, los avisos son en tiempo real dentro de la app y por correo (RF-54…RF-57).

- **Escaneo de QR con cámara.** El retiro se valida buscando el número REC (RF-43). La web solo muestra y descarga el QR del cliente (RF-41, RF-42).

2.2 Sí entra en la web aunque involucre al repartidor (solo lectura o administración)

- Mostrar al cliente el nombre del repartidor asignado y las horas de cada transición (RF-51).
- Ver en admin todos los pedidos con su repartidor (RF-52) y cancelar pedidos (RF-53).
- Reporte de entregas por repartidor (RF-59).
- Promover un cliente a repartidor (RF-61, RPC `promote_to_repartidor`).
- Excluir usuarios repartidor y almacén de los buscadores de clientes (RF-18).
- **Avisar a los repartidores cuando un cliente pide delivery desde la web.** Después de `request_delivery` o `request_delivery_multi`, invocar la Edge Function `send-push-notification` con `{title, body, role: 'repartidor'}`, igual que la app móvil. Sin esto, los repartidores no se enteran de los pedidos hechos en la web.

3. Tecnologías

Capa	Tecnología	Uso / requerimiento
Lenguaje	TypeScript con `"strict": true`	RNF-32
Framework	React + Vite	SPA; código dividido por ruta (RNF-09)
Ruteo	React Router	Rutas protegidas por rol (RF-68)
Acceso a datos	@supabase/supabase-js v2	Auth, consultas, RPC, Realtime
Caché / estado servidor	TanStack Query	Caché, reintentos y paginación (RNF-11, RNF-12, RNF-25)
Formularios	react-hook-form + zod	Validaciones (RF-02, RF-03, RF-14) y protección contra doble envío (RNF-26)
Estilos	Tailwind CSS	Responsivo (RNF-14), identidad de marca del Figma (RNF-19)
Fechas	date-fns + date-fns-tz	Guardar en UTC y mostrar en America/Managua (RF-27, RNF-29)
QR	qrcode.react	RF-41, RF-42 (descarga como PNG)
PDF	@react-pdf/renderer	Recibo descargable e imprimible (RF-40)
CSV	papaparse	RF-63
Calidad	ESLint + Prettier + Vitest + Testing Library	RNF-33
Backend	Supabase existente + Supabase CLI	Migraciones versionadas y tipos generados
Correos	Supabase Edge Function + proveedor de correo (p. ej. Resend) disparado por Database Webhook	RF-56 (fase 2)
Despliegue	Vercel	Preview por PR y producción en main
CI	GitHub Actions	lint, typecheck, test y build en cada PR

Usa la versión estable vigente de cada paquete al crear el proyecto. No agregues ni cambies librerías principales sin acuerdo del equipo (se discute en el PR).

4. Backend existente (Supabase). Resumen inferido de la app móvil

Verifica siempre contra `src/types/database.ts`. Aquí solo aparecen las columnas que usa la app móvil; puede haber más.

4.1 Tablas

Tabla	Columnas relevantes	Notas
users	id, name, last_name, email, phone, address, casillero, role, is_admin, is_available	role ∈ {cliente, repartidor, almacen, admin}. Perfil del cliente (RF-08).
packages	id, user_id, tracking_code, shipping_method_id, package_type_id, weight, price, is_paid, created_at	FK a shipping_methods y package_types. tracking_code único.
package_status_history	package_id, status_id, notes, created_at	Inmutable (RNF-30). notes = ubicación. status_id → status_types(name).
status_types	1 Recibido · 2 En tránsito · 3 En aduana · 4 Listo para recoger	El estado actual es el último registro del historial.
invoices	id, document_type ('recibo'), invoice_number, user_id, subtotal, tax_amount, total, status, has_delivery, delivery_address, delivery_cost, delivery_choice, issued_by, created_at	Número visible: REC- + invoice_number con 5 dígitos. delivery_choice ∈ {pendiente, solicitado, rechazado}.
invoice_items	invoice_id, package_id, description, amount	Se crean junto al recibo, dentro de un RPC.
payments	invoice_id, amount, method, reference, bank_name, paid_at	method ∈ {efectivo, transferencia, tarjeta_datafono, tarjeta_online, contra_entrega}
deliveries	id, client_id, deliverer_id, invoice_id, package_id, delivery_address, status, created_at, accepted_at, on_the_way_at, delivered_at, proof_url	status observados: disponible, aceptado, en_camino, entregado, cancelado. La web no usa deliverer_lat/lng.
delivery_packages	relación pedido ↔ paquetes	Pedido múltiple (RF-48).
user_push_tokens	—	Solo móvil. No usar en la web.

4.2 Funciones RPC (transaccionales)

RPC	Parámetros	¿La usa la web?
create_invoice	p_user_id, p_package_ids[] → id del recibo	Sí (Daviana)
register_payment	p_invoice_id, p_amount, p_method, p_reference, p_bank_name	Sí (Daviana)
request_delivery	p_invoice_id, p_package_id, p_delivery_address	Sí (Alberto) + push a repartidores
request_delivery_multi	p_package_ids[], p_delivery_address	Sí (Alberto) + push a repartidores
decline_delivery	p_invoice_id	Sí (Alberto, RF-49)
promote_to_repartidor	target_user_id, user_phone (opcional)	Sí (Juan, RF-61)
accept_delivery, complete_delivery, update_deliverer_location, delivery_balance	—	**NO** (solo repartidor)
is_almacen()	función security definer usada por RLS	Indirectamente (RLS)

Antes de una solicitud de delivery, la app guarda la dirección y el teléfono en `users` (RF-50). La web debe hacer lo mismo.

4.3 Realtime usado por la app (replicar en web)

- Cambio de rol del usuario conectado: canal sobre `users` filtrado por id. Redirige sin cerrar sesión (RF-10). Si el nuevo rol es repartidor, mostrar la pantalla de app móvil.
- Recibos del cliente (`invoices` por user_id) y seguimiento de paquetes o estados del cliente.
- Pedidos en el panel admin (`deliveries`).

4.4 Auth

- Registro: `auth.signUp({ email, password, options: { data: { name, last_name } } })`. Verificar en las migraciones cómo se crea la fila en `users` y cómo se asigna el casillero (RF-04).
- Recuperar contraseña: `resetPasswordForEmail` con redirect a `/reset-password` de la web. Hay que agregar las URL de la web (local, preview y producción) en Supabase → Auth → Redirect URLs.
- Sesión persistente entre recargas (RF-05) y JWT con renovación automática (RNF-04): comportamiento por defecto de supabase-js.

5. Reglas de negocio

Regla	Valor
Tarifa base por libra	Aéreo $6.50 · Marítimo $3.50 (RF-12)
Multiplicador por categoría	Ropa ×1.00 · Joyería ×1.10 · Electrónica/Tecnología ×1.10 (RF-13)
Fórmula de cotización	precio = peso_lb × tarifa × multiplicador. Peso decimal, mayor que 0 y como máximo 1000 (RF-14)
Costo de delivery	$3.58, se suma al total del recibo al solicitarlo (RF-33)
Número de recibo	REC-00000, correlativo e ininterrumpido. No se elimina; se anula (RF-31, RF-37, RNF-41)
Recibo activo	Máximo uno por paquete (RF-39)
Montos	2 decimales. El cálculo oficial vive en la base de datos (RPC); la web solo muestra (RNF-31)
Centralización	Tarifas y delivery se leen de una tabla `app_config` (nueva, migración de Juan). No escribir estos números fijos en componentes (RNF-34).

5.1 Matriz de permisos (admin vs. almacén)

Función	Administrador	Encargado de almacén
Asignar paquetes nuevos	Bloqueado (ver pendiente 11.2)	Permitido
Estados Recibido y En tránsito	Permitido	Permitido
Estados En aduana y Listo para recoger	Permitido	Bloqueado
Registro de pagos / estado de pago	Permitido	Oculto
Módulo Pedidos (delivery)	Permitido	Bloqueado con candado (RF-66)
Módulo Facturación	Permitido	Bloqueado con candado (RF-66)
Lista de paquetes y línea de tiempo	Todo	Solo paquetes en estados 1–2 (RF-67)

6. Arquitectura y estructura de carpetas

```
semtracar-web/
├─ AGENTS.md           instrucciones para cualquier IA (léelo primero)          (T1 Juan)
├─ CLAUDE.md           remite a AGENTS.md                                       (T1 Juan)
├─ .github/            workflows/ci.yml, pull_request_template.md, CODEOWNERS   (T1 Juan)
├─ docs/
│  ├─ ESTADO.md        avance por RF, bitácora de relevos, decisiones           (todos)
│  └─ brief-semtracar-web.pdf   este documento
├─ supabase/
│  ├─ migrations/      SQL versionado (db pull + nuevas)                        (Juan revisa siempre)
│  └─ functions/       Edge Functions (correo RF-56)                            (T4 Juan)
├─ src/
│  ├─ app/             router, guards por rol, layouts, providers              (T1 Juan)
│  ├─ lib/             supabase.ts, queryClient.ts, format.ts (moneda/fecha)   (T1 Juan)
│  │                   csv.ts                                                  (T2 Daviana)
│  ├─ types/database.ts   GENERADO con supabase gen types. No editar a mano
│  ├─ components/ui/   botones, inputs, tablas, modal, badge, candado, toast   (T1 Juan)
│  └─ features/        en T1 se crean TODAS como esqueleto con página "Próximamente"
│     ├─ auth/ perfil/ acceso/ usuarios/                      (T1 Juan)
│     ├─ paquetes/ facturacion/ retiro/                       (T2 Daviana)
│     ├─ cotizacion/ seguimiento/ recibos/ pedidos/           (T3 Alberto)
│     └─ avisos/ dashboard/                                   (T4 Juan)
│        cada feature:
│        ├─ services/    MODELO: consultas y RPC a Supabase, tipos del dominio
│        ├─ hooks/       CONTROLADOR: useQuery/useMutation, validación, flujo
│        ├─ components/  VISTA: componentes de la feature
│        ├─ pages/       VISTA: pantallas enrutables
│        ├─ routes.tsx   rutas de la feature (con el rol requerido)
│        └─ index.ts     API pública de la feature
└─ .env.example         VITE_SUPABASE_URL=, VITE_SUPABASE_ANON_KEY=
```

- Una feature **no importa archivos internos** de otra; solo lo que exporta su `index.ts`.
- Cada feature puede tener sus propias **consultas de lectura**. Las **escrituras** de un dominio solo las hace la feature dueña (p. ej. pagos solo en `facturacion`).
- El router central (`src/app/router.tsx`) solo importa los `routes.tsx` de cada feature. Así nadie más tiene que editarlo.
- Los componentes no llaman a Supabase directamente: Vista → hook (Controlador) → service (Modelo).

7. Trabajo por turnos (relevo)

7.1 Cómo funciona

Los integrantes no trabajan al mismo tiempo. Cada uno tiene un **turno**: hace su parte, la deja fusionada en `main`, actualiza `docs/ESTADO.md` y entrega el relevo. El siguiente clona o actualiza el repo y su IA sabe qué sigue leyendo **AGENTS.md → docs/ESTADO.md → este brief**. No se necesita ninguna explicación fuera del repositorio.

```
T1 Juan ──cierre──► T2 Daviana ──cierre──► T3 Alberto ──cierre──► T4 Juan (cierre final)
   │  tag turno-1-juan     │  tag turno-2-daviana   │  tag turno-3-alberto   │  tag v1.0.0
   └── docs/ESTADO.md se actualiza al final de cada turno (bitácora + estado de cada RF)
```

Por qué este orden: T1 deja la base, el login y el esqueleto de todas las pantallas. T2 construye el panel que **crea** paquetes y recibos. T3 construye el portal del cliente que **consume** esos datos, así que puede probarlo con datos creados desde la web. T4 agrega lo que depende de todo lo anterior (avisos, correos, dashboard) y hace las pruebas finales.

7.2 Contenido de cada turno

Turno	Requerimientos	Tareas
T1. Juan Base y accesos	RF-01…RF-10, RF-60…RF-62, RF-64…RF-66, RF-68, RNF-34, RNF-42	1) Proyecto Vite + React + TS strict, Tailwind, ESLint/Prettier, Vitest, CI, Vercel y protección de main. 2) `supabase db pull` → migraciones y tipos generados. 3) `components/ui` base según el Figma. 4) Auth completa (registro con términos, login, recuperación, logout), perfil, guards por rol, candados, cambio de rol en tiempo real y pantalla "Tu rol usa la aplicación móvil". 5) Usuarios y roles. 6) Migración `app_config` + servicio de lectura. 7) **Esqueleto de TODAS las features** (carpetas, `routes.tsx` y página "Próximamente" protegida por rol) para que los siguientes turnos solo rellenen. 8) Crear **todos los Issues** con milestone por turno. 9) `AGENTS.md`, `CLAUDE.md` y `docs/ESTADO.md`.
T2. Daviana Operaciones admin/almacén	RF-16…RF-23, RF-29…RF-37, RF-39, RF-43…RF-46, RF-63, RF-67	Paquetes: asignar (código único; buscador sin almacén ni repartidor), listado con filtros, estados con ubicación, historial y pagado. Modo almacén: solo estados 1–2 y sin pago. Facturación: create_invoice, desglose, pagos (register_payment, 5 métodos), saldo y anulación. Retiro por número REC con cobro de saldo. `lib/csv.ts` + exportar paquetes y recibos.
T3. Alberto Portal del cliente	RF-11…RF-15, RF-24…RF-28, RF-38, RF-40…RF-42, RF-47…RF-53	Cotizador (lee app_config). Seguimiento con línea de tiempo y Realtime. Recibos del cliente con `ReceiptPdf` y `ReceiptQr`; agregar también el botón "Descargar PDF" en la facturación del admin, reutilizando el componente. Delivery simple y múltiple, rechazo, contacto en perfil y estado con repartidor; **invocar send-push-notification (role 'repartidor')**. Pedidos del admin con cancelación y exportar CSV con `lib/csv.ts`.
T4. Juan Cierre	RF-54…RF-59, RNF-20, RNF-01 (pruebas)	Tabla `notifications` + triggers, centro de avisos en tiempo real, correos (Edge Function), dashboard y reporte por repartidor. Pruebas de RLS fuera de la UI, pruebas por rol y en 4 navegadores, deploy final, README y tag `v1.0.0`.

7.3 Protocolo de INICIO de turno (lo ejecuta la IA)

1. `git clone https://github.com/juanvega23/semtracar-web` (o `git checkout main && git pull`), `npm ci`, copiar `.env.example` a `.env.local` (el integrante pega los valores, que se comparten por privado y nunca en el repo) y `npm run dev` para comprobar que la app arranca.
2. Leer `AGENTS.md`, luego `docs/ESTADO.md` completo (sobre todo la bitácora del turno anterior y sus "Notas para el siguiente") y luego este brief.
3. Verificar la identidad de Git del integrante (sección 8.1).
4. Confirmar que el turno anterior figura como **Cerrado** y que existe su tag (`git tag`).
5. Listar los Issues del turno (`gh issue list --milestone "T2 · Daviana"`) y las filas de ESTADO.md con ese turno. Proponer al integrante un orden de trabajo y **esperar su confirmación**.
6. Marcar el turno como **En curso** en ESTADO.md (rama `docs/iniciar-turno-N` + PR).

7.4 Protocolo de CIERRE de turno (obligatorio)

1. Todos los PR del turno están fusionados en main. No quedan ramas abiertas del turno.
2. En main pasan lint, typecheck, test y build, y la app arranca con `npm run dev`.
3. Actualizar `docs/ESTADO.md`: estado de cada RF del turno (Hecho / Parcial / No hecho), una entrada en la **bitácora** con qué se hizo, archivos y servicios clave y para qué sirven, decisiones tomadas, desviaciones del brief, migraciones aplicadas, problemas conocidos y **"Notas para el siguiente"**. Cambiar el turno a **Cerrado** y actualizar "Turno actual".

4. Si se resolvió una decisión pendiente (sección 11.2), anotarla en la tabla de decisiones de ESTADO.md.
5. Abrir el PR `docs(estado): cerrar turno N`. Lo aprueba el **siguiente integrante**; así lo lee antes de empezar.
6. Tras fusionarlo, crear el tag: `git checkout main && git pull && git tag turno-2-daviana && git push origin turno-2-daviana`.
7. Avisar al siguiente integrante.

7.5 Reglas del relevo

- Lo que quede incompleto se marca **Parcial** o **No hecho** en ESTADO.md y su Issue queda abierto con la etiqueta `pendiente-relevo`. Nunca se oculta.
- Se puede modificar código de turnos anteriores **solo si es necesario** para la tarea actual (exportar algo en un `index.ts`, corregir un bug). Se hace en commits aparte (`fix(paquetes): …`) y se anota en la bitácora. No se reescribe el trabajo de otro integrante.
- Si un turno se atrasa demasiado, el equipo puede mover tareas al turno siguiente. Se anota en ESTADO.md y se cambia el milestone del Issue.
- Los turnos 1 y 4 son de Juan. Si el equipo decide otro orden, solo se cambia la tabla de turnos en ESTADO.md.

8. Git, GitHub y autoría de commits

Requisito del equipo: **todos los commits de cada integrante deben quedar en el repositorio con su autor real**.

8.1 Configuración (una vez por integrante)

- Juan invita a Alberto y Daviana como colaboradores con permiso de escritura (Settings → Collaborators).
- Cada integrante trabaja en su computadora con **su propia cuenta de GitHub** (SSH o `gh auth login`). La IA nunca usa credenciales de otro integrante.
- Identidad de Git: el correo debe estar **agregado y verificado** en la cuenta de GitHub del integrante, o GitHub no le atribuirá los commits.

```
git config --global user.name  "Nombre Apellido"
git config --global user.email "correo-verificado-en-github@ejemplo.com"
git config user.name && git config user.email     # verificar antes de cada sesión
```

8.2 Ramas

- `main`: protegida y siempre desplegable. Nadie hace push directo.
- Una rama por Issue, creada desde main actualizado: `feat/<modulo>-<descripcion>`, `fix/…`, `docs/…`, `chore/…`. Ejemplos: `feat/seguimiento-timeline`, `fix/auth-redirect-rol`.
- Para traer cambios de main a tu rama usa `git pull origin main` (merge). No hagas rebase de ramas ya subidas.

8.3 Protección de main (Juan la configura en T1)

- Exigir Pull Request con al menos 1 aprobación y que el CI esté en verde.
- Bloquear force push y borrado de main.
- **Método de fusión: solo “Create a merge commit”.** Desactivar “Squash” y “Rebase” en Settings → General. Así cada commit individual y su autor se conservan en main.

8.4 Mensajes de commit

```
tipo(modulo): descripción en imperativo          ← máx. ~72 caracteres
                                                    ← línea en blanco
Detalle opcional. Refs #12

tipos: feat | fix | refactor | style | test | docs | chore
ej.:   feat(seguimiento): agregar línea de tiempo de estados
       fix(auth): redirigir repartidor a pantalla de app móvil
       chore(db): migración inicial obtenida con supabase db pull
```

El autor del commit es siempre el integrante. Si el equipo lo decide, se puede añadir un trailer `Co-Authored-By` de la IA, pero nunca sustituye al autor.

8.5 Pull Requests y revisión

- Título: `[#Issue] descripción`. En la descripción: qué cambia, RF cubiertos, cómo probar (con qué rol), capturas y `Closes #N`.
- Revisor: el **siguiente integrante en la fila** (en T1 revisa Daviana, en T2 Alberto, en T3 Juan y en T4 Daviana). Todo PR que toque `supabase/` requiere además aprobación de Juan (CODEOWNERS).
- Cambios en `package.json`, `src/app/` o `components/ui/` de turnos anteriores se explican en la descripción del PR y en la bitácora.

8.6 Issues y tablero

- Juan crea en T1 todos los Issues (uno por RF o grupo pequeño de RF), asignados al dueño de cada turno.
- Etiquetas: `modulo:*`, `tipo:feat|fix|docs`, `bloqueado`. Milestones: `T1 · Juan`, `T2 · Daviana`, `T3 · Alberto`, `T4 · Juan`.
- Tablero en GitHub Projects: Por hacer → En progreso → En revisión → Hecho.
- Verificar aportes: `git shortlog -sne` y GitHub → Insights → Contributors.

8.7 Cambios en la base de datos

```
supabase migration new <nombre>        # crea supabase/migrations/<timestamp>_<nombre>.sql
# escribir SQL aditivo (no borrar ni renombrar columnas que usa la app móvil)
supabase db push                       # SOLO tras aprobar el PR; lo ejecuta Juan
supabase gen types typescript --linked > src/types/database.ts
```

Recomendado: un segundo proyecto Supabase de **desarrollo** para probar migraciones y RLS antes de aplicarlas al proyecto real que usa la app móvil.

9. Mientras no es tu turno

- **Revisar PRs** del turno en curso (el siguiente en la fila es el revisor). La revisión también queda registrada en GitHub como aporte.
- Preparar tu turno: leer este brief, ESTADO.md y el Figma de tus pantallas, y probar la app en el preview de Vercel.
- Reportar bugs como Issues (etiqueta bug) asignados al dueño del turno en curso.
- No hacer commits sobre las tareas del turno de otro integrante.

10. Definición de “terminado” (cada PR)

- Cubre los RF indicados en el Issue y se probó con cada rol afectado (cliente, admin, almacén y repartidor si aplica).
- lint, typecheck, test y build pasan. Sin any injustificado.

- Estados de carga, vacío y error con mensajes comprensibles en español (RNF-16, RNF-17). Botones de escritura deshabilitados mientras se envía (RNF-26).
- Responsivo en móvil, tableta y escritorio. Fechas en hora de Managua y montos en USD con 2 decimales.
- Sin secretos ni `console.log` con datos personales (RNF-35, RNF-40).
- Commits pequeños con autor correcto y PR aprobado por otro integrante.
- Al cerrar el turno, además: `docs/ESTADO.md` actualizado y tag creado (sección 7.4).

11. Decisiones tomadas y pendientes

11.1 Tomadas

- El repartidor no usa la web: se muestra la pantalla “Tu rol usa la aplicación móvil” y se cierra la sesión.
- Sin mapa de entrega en la web.
- Sin push en la web; la web sí dispara el push a repartidores al crear pedidos.
- QR: solo mostrar y descargar. La validación de retiro es por número REC.
- Mismo proyecto Supabase que la app móvil; cambios de BD solo con migraciones.
- Fusión de PRs con merge commit para conservar todos los commits y sus autores.
- Trabajo por turnos secuenciales: Juan → Daviana → Alberto → Juan (cierre), con el avance en docs/ESTADO.md.

11.2 Pendientes (la IA debe preguntar antes de implementar lo afectado)

Primero revisa la tabla de decisiones de `docs/ESTADO.md`. Si un punto ya está resuelto ahí, aplica esa resolución.

- **RF-29:** el requisito dice que el recibo se genera automáticamente al llegar a “Listo para recoger”, pero la app actual lo crea a mano el admin agrupando paquetes (create_invoice). Decidir: trigger en BD o ajustar el requerimiento.
- **Matriz:** el admin aparece “Bloqueado” para asignar paquetes nuevos. Confirmar si es intencional.
- **Impuesto:** confirmar tasa y cálculo de `tax_amount` en create_invoice.
- **Nombre visible:** “Semtracar” (documentos) o “Nica Cargo Xpress” (app).
- **Avisos:** diseño de la tabla `notifications` y qué eventos la alimentan.
- **Proyecto Supabase de desarrollo:** crear uno o no.

12. Prompt de arranque (cada integrante lo pega en su IA al empezar su turno)

```
Soy <Juan | Alberto | Daviana>, integrante del equipo Semtracar Web, y me toca mi turno.
Repositorio: https://github.com/juanvega23/semtracar-web

1) Clona o actualiza main e instala dependencias (te pasaré los valores de .env.local).
2) Lee en este orden: AGENTS.md, docs/ESTADO.md y docs/brief-semtracar-web.pdf.
   ESTADO.md dice qué está hecho y qué turno sigue; el brief tiene las reglas.
3) Verifica que git config user.name / user.email sean los míos: <nombre> / <correo>.
4) Confirma que el turno anterior está Cerrado y dime en máximo 10 líneas:
   qué dejó hecho el turno anterior, qué me toca a mí (RF e Issues) y en qué
   orden propones hacerlo. Espera mi confirmación antes de escribir código.

Reglas: una rama por Issue, commits pequeños (sección 8.4), sin push a main,
sin tocar supabase/ sin avisarme. Si algo de la sección 11.2 me afecta, pregúntame.
Cuando todo mi turno esté fusionado, ejecuta el protocolo de cierre (sección 7.4).
```

Para el **turno 1** (Juan) el repositorio está vacío: la IA no encontrará AGENTS.md ni ESTADO.md. Juan los sube primero desde las plantillas que acompañan este brief, junto con este PDF en `docs/`.